Lab 3 report: what you actually need to do

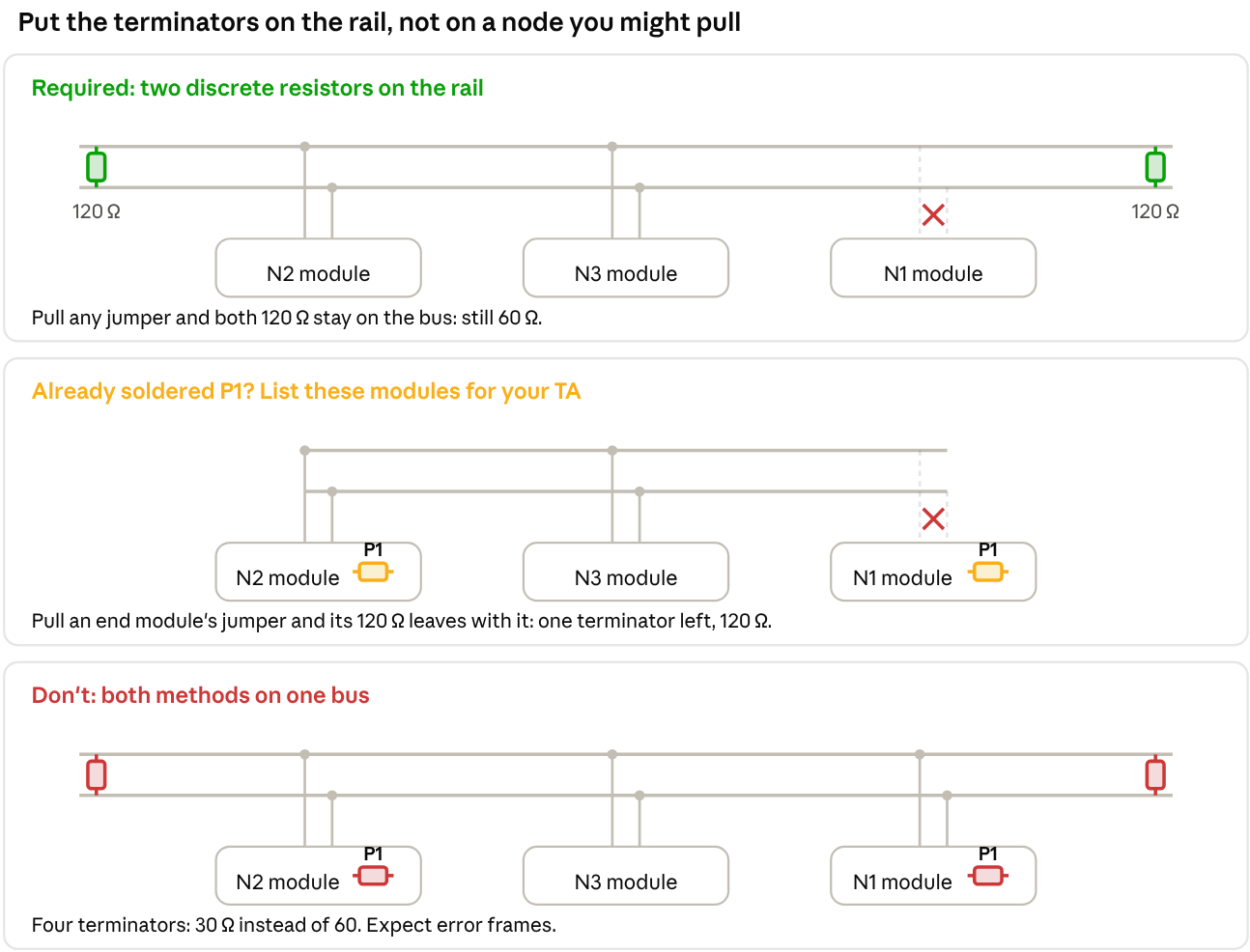
Oct 22, 2026 · @Ayaan Shaikh

What each part of the Lab 3 report needs, what you measure, and what checkoff tests. Where this differs from the handout, follow this page.

1. Wiring schematic of the dual bus

Extend your Lab 2 schematic: both CAN buses with termination, every XIAO module's SCK, MOSI, MISO, CS and INT pins, the common ground, and your test-point boards.

Termination: use two discrete 120 Ω resistors per bus, one at each end of the rail, with every P1 pad open, so pulling a jumper never removes termination. Check for 60 Ω across CANH-CANL with power off.



termination: recommended, allowed, and wrong · one bus shown

If you already soldered P1: pulling that module's jumper also removes a terminator, which hurts fault tolerance. **Clearly list which modules you terminated** in your report and tell your TA, so we don't pull those. Don't also add discrete resistors on that bus.

2. Task communication graph figure

You draw **one** graph, starting from the handout's Figure 3. Make sure it has:

- **Every task in the Part 2 task table**, including TEST_BTN on N2 and N3
- **Every message in the catalog**, including FAULT (0x020)
- **Each CAN arrow labeled with its ID and period**; every frame goes on both bus A and bus B
- **The four chains highlighted:** brake/throttle, steering, blinker, error propagation

3. Single-bus analysis

We cut the handout's roughly 14 bench measurements to 6, all required: 4 here, 2 in Part 4. Include a scope screenshot of each; the guide at the end shows how. For everything else, use the placeholder C values in the Part 2 task table.

- **This part is single bus only. No redundancy, no relay; that is Part 4.**
- **Compute R for every task on N2 and N3**, using the formulas below. Show your work for every row.
- **Then add up the five chains** in the Part 3.3 table and compare each total to its deadline.
- **Two different deadlines.** Check each task's R against its own D, and each chain's total against the chain deadline.
- **Missing a deadline is OK.** Mark it, say why it misses, and say what would fix it.

Quick reminder of the letters

- **C**: the task's own execution time. For a CAN frame, its time on the bus: 270 µs.
- **T**: the period, or minimum time between triggers. **D**: that task's own deadline.
- **B**: blocking, the time spent waiting on a lower-priority task that holds the SPI bus. It is one frame load (#4), counted once; 2 × #4 if your send helper holds the SPI lock across both buses; 0 for the lowest-priority task.
- **R** (task): R = C + B + the sum, over each higher-priority task j, of ⌈R / T_j⌉ × C_j. Start with R = C + B and repeat until R stops changing.

- **J** (message): how late the frame is queued after its trigger, i.e. the sending task's R. For CTRL it is the Pi stage (#1).
- **w** (message): how long the frame waits for the bus: w = 270 µs of blocking + the sum, over each higher-priority message j, of ⌈(w + J_j + 2 µs) / T_j⌉ × C_j.
- **R** (message): R = J + w + C.

Measurements for this part. #1 is the measured Pi latency the handout asks for.

#	Number	Your max (µs)	Screenshot
1	Pi stage		
2	CAN_RX C, N3		
3	DRIVE C, N3		
4	SPI frame load (B)		

Node RTA table: N3

Task	Priority	C (µs)	T	D	B (µs)	R (µs)	R ⩽ D?
CAN_RX		**#2 (YOURS)**	20 ms per message, 7 per bus	0.3 ms	**#4 (YOURS)**		
DRIVE		**#3 (YOURS)**	20 ms	0.5 ms	**#4 (YOURS)**		
BLINK_R		20	20 ms	0.5 ms	**#4 (YOURS)**		
TEST_BTN		20	50 ms	10 ms	**#4 (YOURS)**		
PID		200	10 ms	10 ms	**#4 (YOURS)**		
HB_TX		120	20 ms	20 ms	**#4 (YOURS)**		
HB_MON		30	20 ms	20 ms	**#4 (YOURS)**		

Node RTA table: N2

Task	Priority	C (µs)	T	D	B (µs)	R (µs)	R ⩽ D?
CAN_RX		80	20 ms per message, 7 per bus	0.3 ms	**#4 (YOURS)**		
STEER		50	20 ms	1 ms	**#4 (YOURS)**		
BLINK_F		20	20 ms	0.5 ms	**#4 (YOURS)**		
TEST_BTN		20	50 ms	10 ms	**#4 (YOURS)**		
FORCE_TX		120	50 ms	50 ms	**#4 (YOURS)**		

Task	Priority	C (µs)	T	D	B (µs)	R (µs)	R ⩽ D?
HB_TX		120	20 ms	20 ms	**#4 (YOURS)**		
HB_MON		30	20 ms	20 ms	**#4 (YOURS)**		

Message RTA table. Only CTRL is required; it is the same on both buses. Utilization is not graded here.

Message	J (µs)	C (µs)	B (µs)	w (µs)	R (µs)
0x010 CTRL	**#1 (YOURS)**	270	270		

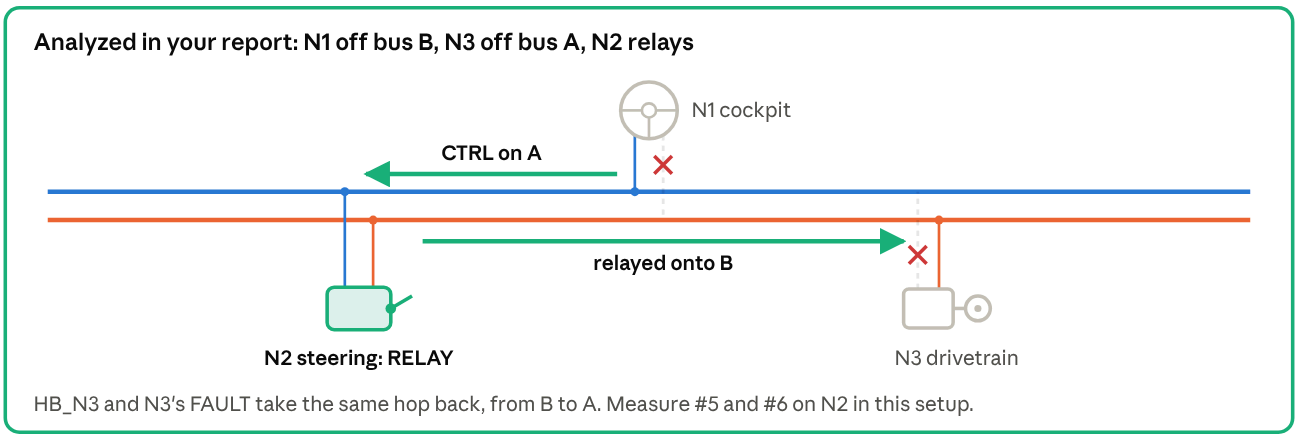
End-to-end chain table

Chain	Stages to add up	Total	Deadline	Met?
Brake / throttle	Pi + CTRL on bus + CAN_RX (N3) + DRIVE		2 ms	
Steering	Pi + CTRL on bus + CAN_RX (N2) + STEER + one servo period		50 ms	
Blinker start	Pi + BLINK on bus + CAN_RX + BLINK_R		100 ms	
Blinker sync	N3 receive path minus N2 receive path, worst case		1 ms	
Error propagation	TEST_BTN + 3 missed heartbeats (66 ms) + one HB_MON period (20 ms) + HB_MON		100 ms	

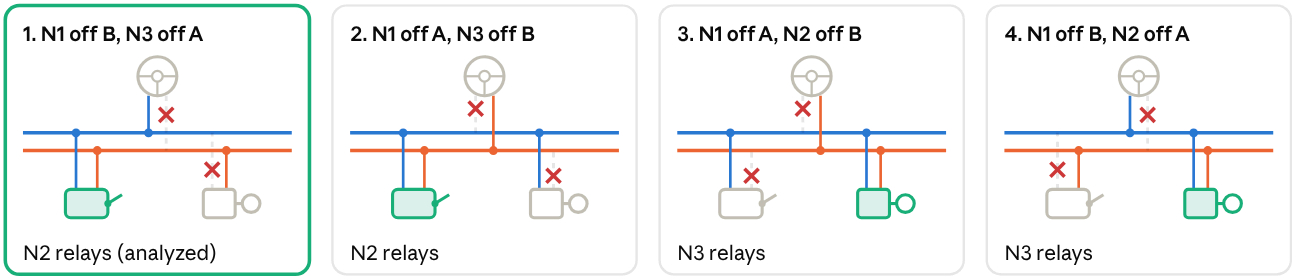
4. Dual-bus and relay analysis

Add redundancy and analyze one relay case: N1 off bus B, N3 off bus A, so N2 relays.

Your report analyzes one relay case; checkoff tests one of four



At checkoff the TA picks one of these four. The relay always runs on an STM32.



Wheel = N1 cockpit (Pi), servo = N2 steering, motor = N3 drivetrain.
Not tested: N2 and N3 split across the two buses. Only the Pi would be left on both, and a relay on Linux can't be bounded.



the relay case you analyze, and the four checkoff cases

We don't test a relay through N1: it would put Linux in the relay path, which has no worst-case timing bound.

- **2 more measurements** on N2: CAN_RX C and RELAY C.
- **Add RELAY to your task graph.** It is its own thread on N2. Draw RELAY and the frames it forwards; don't redraw the rest.
- **Defend your relay design**: who relays and when, how you avoid loops, and RELAY's priority. There are several valid designs.

Measurements for this part

#	Number	Your max (µs)	Screenshot
5	CAN_RX C, N2		
6	RELAY C, N2		

Re-analysis table: N2 with RELAY

Task	Priority	C (µs)	T	D	B (µs)	R (µs)	R ⩽ D?
CAN_RX		**#5 (YOURS)**	20 ms per message, 7 per bus	0.3 ms	**#4 (YOURS)**		
RELAY		**#6 (YOURS)**	every frame N2 forwards	on the brake chain	**#4 (YOURS)**		
STEER		50	20 ms	1 ms	**#4 (YOURS)**		
BLINK_F		20	20 ms	0.5 ms	**#4 (YOURS)**		
TEST_BTN		20	50 ms	10 ms	**#4 (YOURS)**		
FORCE_TX		120	50 ms	50 ms	**#4 (YOURS)**		
HB_TX		120	20 ms	20 ms	**#4 (YOURS)**		
HB_MON		30	20 ms	20 ms	**#4 (YOURS)**		

Bus utilization for your relay design

Start from 8.6% and add each frame your design relays onto that bus: 270 µs divided by its period (one extra frame every 20 ms adds 1.35%). Show your work: list each relayed frame and its period.

Bus	New utilization with your relay	⩽ 50%?
A		
B		

Relayed CTRL on bus B

Message	J (µs)	C (µs)	B (µs)	w (µs)	R (µs)
0x010 CTRL, relayed	Fill in: Pi (#1) + CTRL on A (540) + CAN_RX R (N2) + RELAY R (N2) =	270	270		

J is the whole first hop. Show the sum. The relayed brake chain is then this row's R + CAN_RX (N3) + DRIVE; don't count the first hop twice.

Chains in scenario (c): verdict and what changed

Chain	Stages to add up	Total	Deadline	Met?	What changed
Brake / throttle	Pi + CTRL on A + CAN_RX (N2) + RELAY + CTRL on B + CAN_RX (N3) + DRIVE		2 ms		
Steering			50 ms		
Blinker start			100 ms		
Blinker sync			1 ms		
Error propagation			100 ms		

5. Fault-injection matrix (R4.8 a–f)

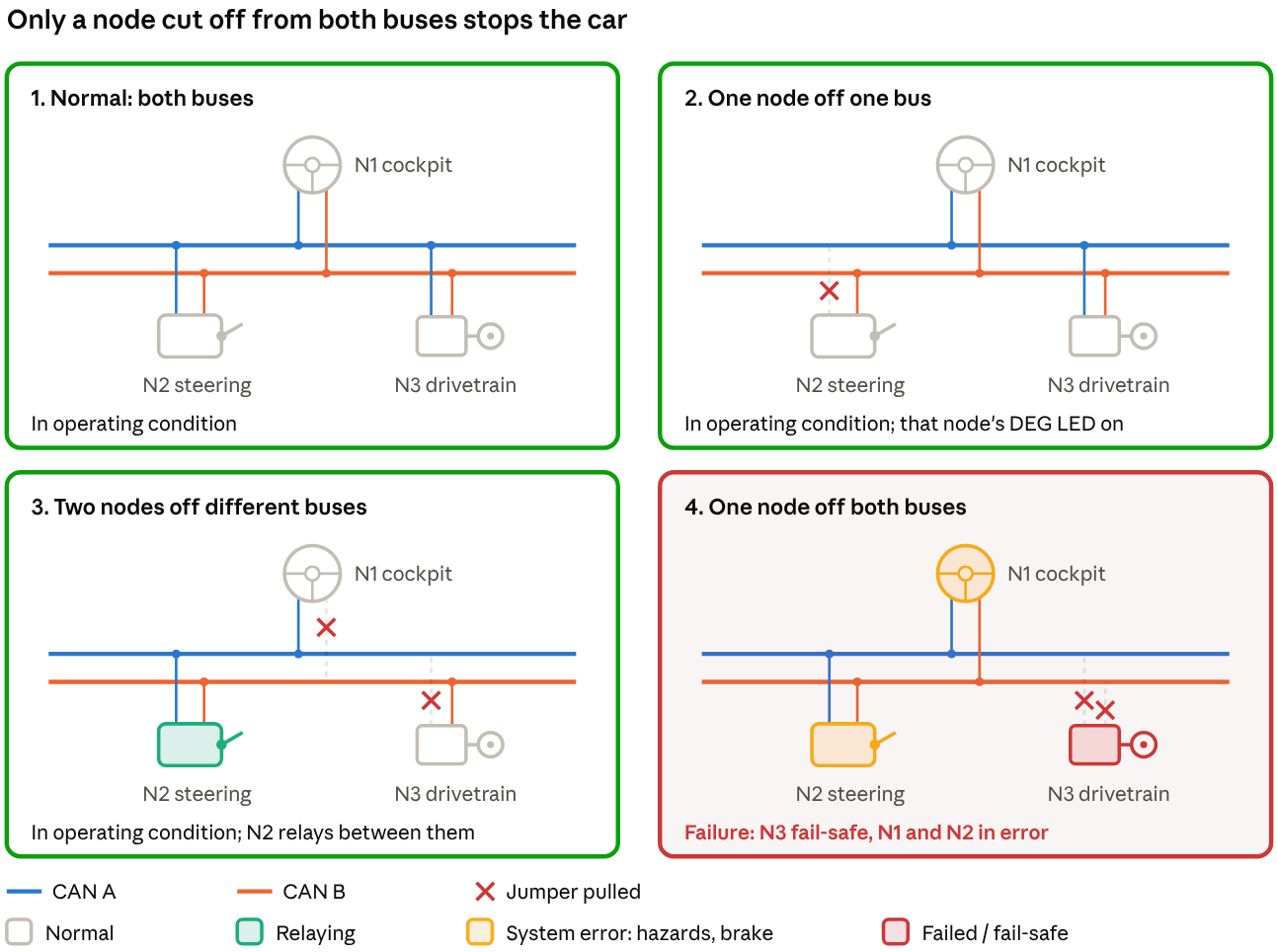
Unchanged from the handout.

6. Updated team-defined values table and Lab 1 revision history

Unchanged from the handout.

Checkoff

When we pull CAN jumpers, your network is in one of four states. Only state 4 should fail. Pulling every node off one bus counts as state 2.

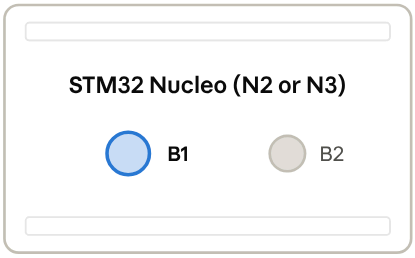


the four communication states at checkoff

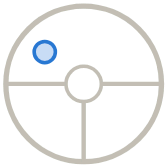
Two other ways into failure.

- **Test button.** Press the blue USER button on an STM32 (not RESET), or the wheel's test button for the Pi. That node sends one FAULT and stops its heartbeats, and the other nodes go into error. A double press within 500 ms recovers it.
- **Bad input.** A CTRL value outside your valid ranges puts the receiving zone into error and sends FAULT.

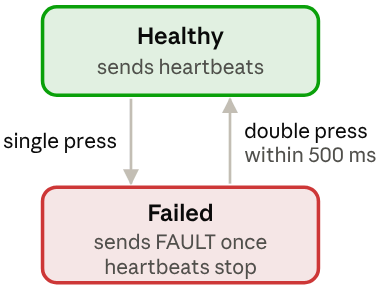
One press fails the node; a double press brings it back



Blue USER button (B1): press this
Black RESET (B2): not this, it reboots



Wheel test button (Pi)



The other nodes see the FAULT or the missing heartbeats and go to system error within 100 ms: hazards on, N3 brakes.
After a double press, heartbeats resume and the whole car returns to normal.

test button: fail and recover one node

Suggested guide: how to take the 6 measurements

Each measurement needs a scope screenshot in your report showing both probes and the max.

#	Number	Node	Start probe	Stop probe
1	Pi stage	N1	UDP_RX, toggled right after the socket read returns	CMD_TX, toggled right after the first CAN write returns
2	CAN_RX C	N3	INT line of the CAN A module falling	CAN_RX_A, toggled when dispatch first gets the frame
3	DRIVE C	N3	CAN_RX_A	DIR_A changing on a brake frame
4	SPI frame load (B)	N2 or N3	Module CS line going low	CS going high again (longest low window you see)
5	CAN_RX C	N2	INT line of the CAN A module falling	CAN_RX_A on N2
6	RELAY C	N2	CAN_RX_A on N2, with N1 off B and N3 off A	RELAY_TX, toggled after the send on bus B returns

- Run at least a minute of normal traffic and report the **max**, not the average.

- For #2 to #6, keep the node's other tasks idle so nothing preempts the task you are measuring.
- A GPIO toggle costs a few µs in C and tens of µs in Python; say which you used.
- Send with a callback on the STM32s. Without one, `can_send()` waits until the frame is on the bus and your pulse includes bus time.